

CHAPTER 5: SOFTWARE TESTING TECHNIQUES AND MAINTENANCE

5.1 TESTING FUNDAMENTALS

❖ SOFTWARE TESTING:

- Software Testing is a process of verifying and validating whether the Software Product or Application is working as expected or not.

Once source code has been generated, software must be tested to uncover as many errors as possible before delivery to your customer. Your goal is to design a series of test cases that have a high likelihood of finding errors. Software testing techniques enter the picture that provide systematic guidance for designing tests that exercise the internal logic of software components, and exercise the input and output domains of the program to uncover errors in program function, behavior and performance.

Software testing principles guide effective testing by focusing on requirement traceability, early planning, defect detection, independence, and efficient use of resources.

The major software testing principles can be listed as:

- All tests should be traceable to customer requirements.
- Tests should be planned long before testing begins.
- Destructive rather than constructive step.
- Testing should begin “in the small” and progress toward testing “in the large.”
- Exhaustive or complete error free testing is not possible.
- To be most effective, testing should be conducted by an independent third party.
- A good test is not redundant because testing time and resources are limited.
- A good test case is one that has a high probability of finding an as-yet-undiscovered error.
- A good test should be neither too simple nor too complex.

Testing is a set of activities that can be planned in advance and conducted systematically. During early stages of testing, a software engineer performs all tests. However, as the testing process progresses, testing specialists may become involved.

Software Testing Can be Divided into Two Steps: Verification and Validation

Software testing is one element of a broader topic that is often referred to as verification and validation (V&V).

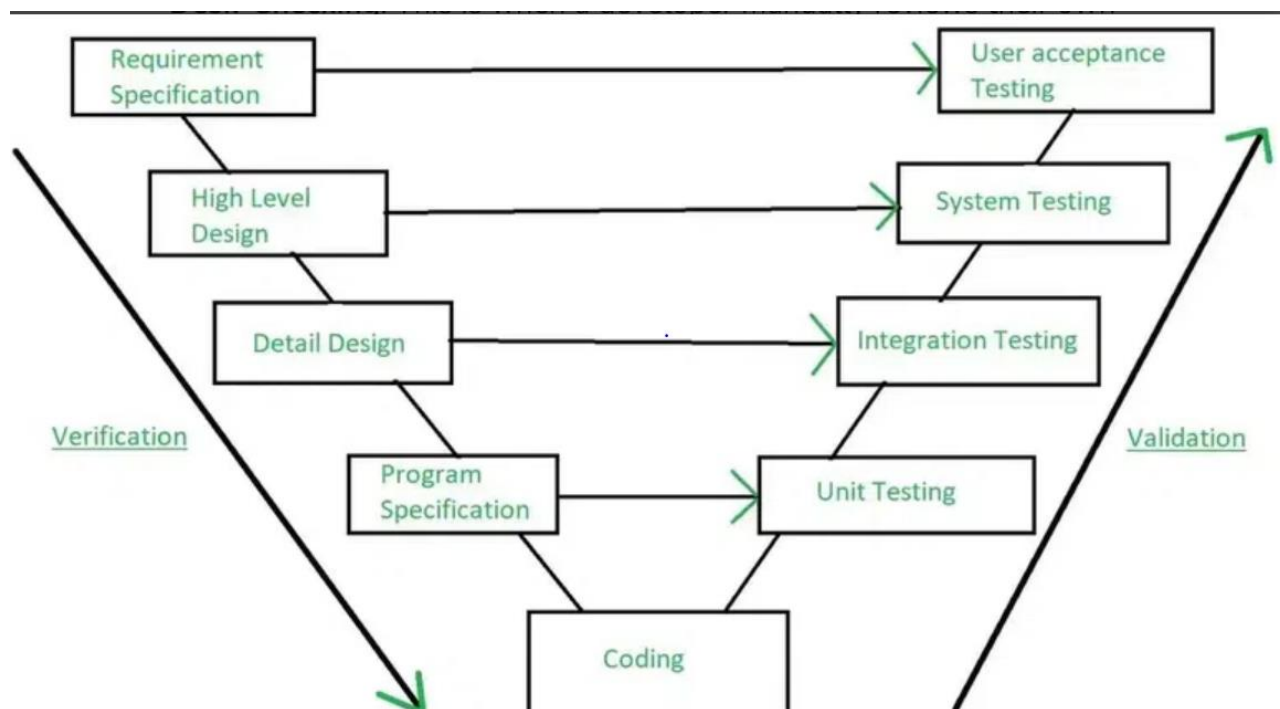
Verification refers to the set of tasks that ensure that software correctly implements a specific function. Verification checks whether the software is built according to specifications and design.

Validation refers to a different set of tasks that ensure that the software that has been built is traceable to customer requirements. Validation checks whether the software meets customer needs and requirements.

Boehm [Boe81] states this another way:

Verification: “Are we building the product right?”

Validation: “Are we building the right product?”



Verification	Validation
Ensures the product is being built correctly by evaluating processes and artifacts.	Ensures the product meets user needs and requirements by evaluating the actual software.
Are we building the product right?	Are we building the right product?
Process-oriented.	Product-oriented.
Performed during the development process (early stages).	Performed after the product is developed (final stages).
Static testing (without executing the code).	Dynamic testing (executing the code).
Requirement management tools (e.g., JAMA), design tools.	Testing frameworks, simulators (e.g., Selenium, JMeter).
Performed by Developers, Business Analysts, QA teams.	Performed by QA teams, end-users, stakeholders.
Confirms that the product complies with requirements and design specifications.	Confirms that the product works as intended in a real-world scenario.
Preventive in nature (identifies errors early).	Detective in nature (identifies errors post-development).

❖ **VALIDATION TESTING:**

- The process of evaluating software during the development process or at the end of the development process to determine whether it satisfies specified business requirements.
- Validation Testing ensures that the product actually meets the client's needs. It can also be defined as to demonstrate that the product fulfills its intended use when deployed on appropriate environment.

Key Objectives of Validation Testing:

- Confirm that the software meets business requirements.
- Ensure the software fulfills user needs and expectations.
- Validate that the product operates under real-world conditions.

Example of Validation Testing

Let's consider an e-commerce website that allows users to purchase products online. **Validation testing** would involve checking whether the checkout process works smoothly, whether payment methods are functioning, and whether the final **product meets** the user's needs. This might include **UAT**, where real customers simulate purchases to verify the system is user-friendly and performs as expected.

Benefits of Validation Testing:

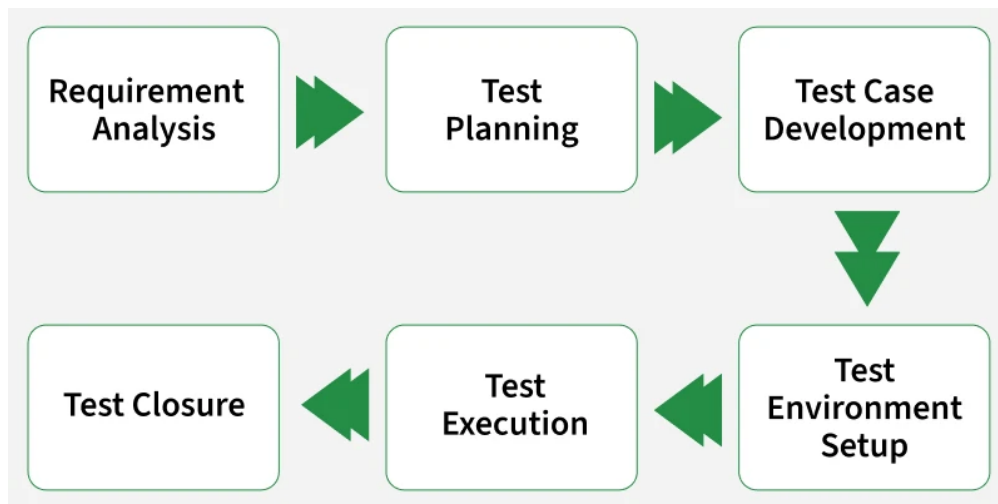
- **Meets User Needs:** Confirms the software solves real-world problems and aligns with customer expectations, improving user experience and adoption.
- **Reduces Costs:** Identifies defects and gaps early, preventing expensive rework and preventing a faulty product from reaching the market.
- **Increases Quality & Reliability:** Catches issues missed during development, leading to a more stable, robust, and dependable final product.
- **Boosts User Confidence:** Accurate, reliable systems build trust and satisfaction, encouraging continued use.
- **Ensures Compliance:** Helps meet industry standards and regulations, avoiding penalties and reputational damage, especially in sectors like healthcare or finance.
- **Improves Efficiency:** By finding issues sooner, teams spend less time on bug fixing later in the cycle, speeding up releases.
- **Validates Requirements:** Confirms that requirements are accurate, complete, and match stakeholder expectations, preventing scope creep and misunderstandings.
- **Provides Business Confidence:** Gives stakeholders assurance that the product is fit for purpose and ready for launch.

❖ SOFTWARE TESTING LIFE CYCLE (STLC):

Software Testing Life Cycle (STLC) is the testing process which is executed in systematic and planned manner. STLC is a process that verifies whether the Software Quality meets the expectations or not. In STLC process, different activities are carried out to improve the quality of the product.

Phases of STLC:

There are six major phases of the Software Testing Life Cycle (STLC) that we are discussing here in details.



1. Requirement Analysis

Requirement Analysis is the first phase of the Software Testing Life Cycle (STLC). In this phase, the QA team analyzes the requirements to identify what needs to be tested. If any requirement is missing, unclear, or ambiguous, the QA team communicates with stakeholders to gain a clear understanding.

Deliverables: Testable requirements, Requirement Traceability Matrix (RTM).

2. Test Planning

Test Planning is the most important phase of STLC, where the overall testing strategy is defined. In this phase, the test manager estimates effort, cost, resources, tools, risks, and schedule. This phase starts after requirement analysis is completed.

Deliverables: Test Plan document, effort estimation.

3. Test Case Development

The Test Case Development phase begins after test planning. In this phase, the testing team designs detailed test cases and prepares test data. Test cases are reviewed by the QA team to ensure completeness and correctness. The main activities performed during the Test Case Development phase include:

- Identifying the test cases that will be developed
- Writing test cases that are clear, concise and easy to understand

- Creating test data and test scenarios that will be used in the test cases
- Identifying the expected results for each test case
- Reviewing and validating the test cases
- Updating the requirement traceability matrix (RTM) to map requirements to test cases.

4. Test Environment Setup

Test Environment Setup is a critical phase of STLC. It defines the hardware, software, network, and tools required for testing. This activity can be done parallel to test case development. Usually, the environment is set up by developers or system administrators, not the QA team.

Deliverables: Test environment ready, smoke test results.

5. Test Execution

After test case development and environment setup, the test execution phase begins. The testing team executes test cases and compares expected results with actual results. If defects are found, they are logged and retested after fixes.

Deliverables: Defect reports, test execution results.

6. Test Closure

Test Closure is the final phase of STLC, where the testing process is evaluated and completed. The final activities carried out during the Test Closure phase include:

- Prepare a Test Summary Report (test cases executed, pass/fail count, defects found/resolved).
- Ensure all defects are tracked and closed.
- Clean up the test environment.
- Archive test cases, data and reports.
- Conduct a retrospective for lessons learned.
- Share knowledge with stakeholders.

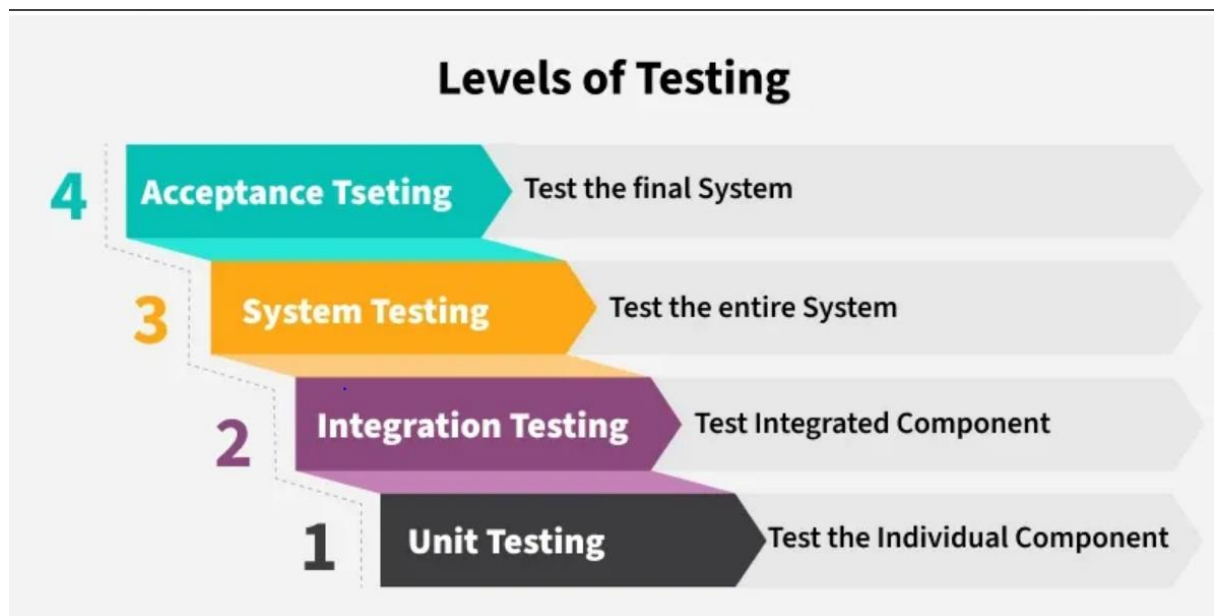
Why is Importance of Software Testing?

Software testing is important for the several reasons and it mainly classified in the following reason which are affecting the most.

- **Defects can be Identified Early:** Software testing is important because if there are any bugs they can be identified early and can be fixed before the delivery of the software.
- **Improves Quality of Software:** Software Testing uncovers the defects in the software, and fixing them improves the quality of the software.
- **Increased Customer Satisfaction:** Software testing ensures reliability, security, and high performance which results in saving time, costs, and customer satisfaction.
- **Helps with Scalability:** Software testing type non-functional testing helps to identify the scalability issues and the point where an application might stop working.
- **Saves Time and Money:** After the application is launched it will be very difficult to trace and resolve the issues, as performing this activity will incur more costs and time. Thus, it is better to conduct software testing at regular intervals during software development.

❖ LEVEL OF SOFTWARE TESTING:

Levels of Testing divided into four main levels, which are as follows:



1. Unit Testing:

Unit testing is the process of testing the smallest parts of your code, like it is a method in which we verify the code's correctness by running one by one. It's a key part of software development that improves code quality by testing each unit in isolation. The goal here is to catch any issues early before those small components are integrated with the rest of the system.

Mainly features are:

- Testing individual functions or components.
- Ensuring correct behavior at the smallest level.
- Catching errors early.

2. Integration Testing:

This level checks how different modules or components of the software work together. It's important because even if individual parts work perfectly, they might face issues when interacting with one another.

Integration testing ensures that data flows correctly between modules and that they communicate seamlessly. It helps catch problems that might arise when different parts of the system interact.

Types of Integration Testing

1. Big bang testing:

- As the name suggests, big bang form of testing all the modules is combined to form a complete system and then tested for bugs.

2. Top-down testing:

- It is a method in which integration testing takes place from top to bottom following the control flow of software system.
- The higher-level modules are tested first and then lower-level modules are tested and integrated in order to check the software functionality.

3. Bottom-up testing:

- It is a method in which the lower-level modules are tested first.
- These tested modules are then further used to facilitate the testing of higher-level modules. The process continues until all modules at top level are tested.

3. System Testing

This stage checks whether the entire system functions as expected in a real-world environment. It includes both functional and non-functional tests to ensure that the software meets customer needs.

- Full end-to-end testing of the software.
- Verifying both functional and non-functional requirements.
- Testing the software's behavior in real-world conditions.

Types of System Testing

i. Usability Testing:

- To make sure that the system is easy to use, learn and operate.

ii. Load Testing:

- Is necessary to know that a software solution will perform under real-life loads

iii. Regression Testing:

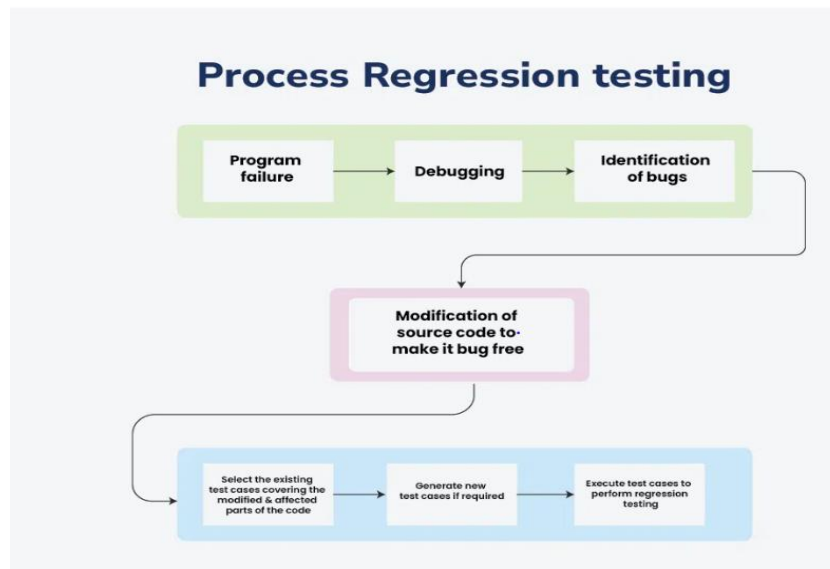
- Involves testing done to make sure none of the changes made over the course of the development process have caused new bugs.

It also makes sure no old bugs appear from the addition of new software modules over time.

- **Regression Testing** involves re-executing a previously created test suite to verify that recent code changes haven't caused new issues. This verifies that updates, bug fixes, or enhancements do not break the functionality of the application.

Regression testing is necessary in several scenarios to maintain software quality:

- When new functionality is added to the system and the code has been modified to absorb and integrate that functionality with the existing code.
- When some defect has been identified in the software and the code is debugged to fix it.
- When the code is modified to optimize its working.



- **Identify Code Changes:** Analyze the source code to determine which areas have been modified, such as new features, bug fixes, or optimizations.
- **Debug and Fix Failures:** If existing test cases fail due to changes, debug the code to identify and resolve defects.
- **Modify Code:** Apply necessary updates to the code to incorporate changes or fixes.
- **Select Test Cases:** Choose relevant test cases from the existing test suite that cover modified and affected areas. Add new test cases if needed to address new functionality.
- **Execute Regression Tests:** Run the selected test cases, either manually or using automated tools, to verify system behavior.
- **Analyze Results:** Review test outcomes to identify regressions, document issues, and recommend fixes.
- **Retest as Needed:** If defects are found, fix them and re-run tests to confirm resolution.

iv. Recovery Testing:

- To make sure how well the system recovers from various input errors and other failure situations.

v. Migration Testing:

- Is done to ensure that the software can be moved from older system infrastructures to current system infrastructures without any issues.

vi. Functional Testing:

- It involves trying to think of any possible missing functions. Testers might make a list of additional functionalities that a product could have to improve it during functional testing.

vii. Documentation Testing:

- To make sure that the system's user guide and other help topics documents are correct and usable.

viii. Interoperability Testing:

- To make sure whether the system can operate well with third-party products or not.

ix. Performance Testing:

- To make sure the system's performance under the various condition, in terms of performance characteristics
- It also makes sure no old bugs appear from the addition of new software modules over time.

x. Reliability Testing:

- To make sure the system can be operated for a longer duration without developing failures.

4. Acceptance Testing:

Acceptance testing, also known as User Acceptance Testing(UAT), is the final test before releasing the software to the end-users. In this phase, the customer or end-users verify if the software meets their needs and expectations.

UAT is crucial because it verify the software is ready for production. It's the last chance to catch any overlooked issues before deployment. If the software passes this stage, the customer gives the green light for release.

- Validating the software against business requirements.
- Ensuring the software meets customer expectations.
- Getting final approval from the customer.

Types of Acceptance Testing:

a. User Acceptance Testing (UAT):

- User acceptance testing is used to determine whether the product is working for the user correctly.
- Specific requirements which are quite often used by the customers are primarily picked for testing purposes. This is also termed as End-User Testing.

b. Operational Acceptance Testing (OAT):

Operational Acceptance Testing is a non-functional testing phase that verifies whether a software system is operationally ready, reliable, and maintainable for deployment in the production environment.

c. Business Acceptance Testing (BAT):

- BAT is used to determine whether the product meets the business goals and purposes or not.
- BAT mainly focuses on business profits which are quite challenging due to the changing market conditions and new technologies, so the current implementation may have to be changed which results in extra budgets.

d. Alpha Testing:

- It is a type of acceptance testing; performed to identify all possible issues and bugs before releasing the final product to the end users.
- Alpha testing is carried out by the testers who are internal employees of the organization.
- The main goal is to identify the tasks that a typical user might perform and test them.

e. Beta Testing:

- It is performed by “real users” of the software application in “real environment” and it can be considered as a form of external User Acceptance Testing.
- It is the final test before shipping a product to the customers. Direct feedback from customers is a major advantage of Beta Testing. This testing helps to test products in customer’s environment.
- Beta version of the software is released to a limited number of end-users of the product to obtain feedback on the product quality. Beta testing reduces product failure risks and provides increased quality of the product through customer validation.

5.2 TEST CASE DESIGN

A Test Case is a step-by-step guide used to check if a specific part of a software application works correctly as expected. It defines the actions to be taken, the required data, and the expected result to validate that a feature functions as per the requirements.

Test case design is the systematic process of defining and documenting a detailed set of conditions, inputs, steps, and expected outcomes to verify that a software application or a specific feature works correctly according to its requirements.

Key component of test case:

Fields	Description
Test Case ID	Each test case should have a unique ID.
TestCase Description	Each test case should have a proper description to let testers know what the test case is about.
Pre-Conditions	Conditions that are required to be satisfied before executing the test case.
Test Steps	Mention all test steps in detail and to be executed from the end-user's perspective.
Test Data	Test data could be used as input for the test cases.
Expected Result	The result is expected after executing the test cases.
Post Condition	Conditions need to be fulfilled when the test cases are successfully executed.
Actual Result	The result that which system shows once the test case is executed.
Status	Set the status as Pass or Fail on the expected result against the actual result.
Reference Document	Mention the path of the reference document.

Key Steps in Test Case Design:

The process of designing effective test cases typically involves several stages:

1. **Analyze Requirements:** Thoroughly understand all functional and non-functional requirements from documentation like user stories or design specifications. This forms the foundation for all test cases.
2. **Determine Test Scope and Objectives:** Define what aspects of the software will be tested and the goals of the testing effort (e.g., verifying a specific feature, ensuring security).
3. **Identify Test Scenarios:** Based on the requirements, outline high-level possibilities or conditions the user might encounter. For example, a "login" scenario could include possibilities for valid and invalid credentials.
4. **Select Test Design Techniques:** Choose appropriate methodologies to systematically derive specific test cases from the scenarios.
5. **Write Detailed Test Cases:** Document each test case with a unique ID, clear steps, test data, and the expected result.
6. **Review and Baseline:** Have test cases peer-reviewed by team members or business analysts to ensure accuracy and completeness. Update them regularly as the software evolves.

Example: Peer review identifies that the test case does not include “forgot password” link functionality.

7. **Prepare Test Data and Environment:** Gather or create the necessary data inputs and set up the appropriate testing environment (hardware, software, network configuration) before execution.

Key Test Case Design Techniques:

Testers use various techniques to design effective and efficient test cases, broadly categorized into black-box (based on external specifications) and white-box (based on internal code logic) methods.

1. Black-Box Techniques

Black Box Testing is a software testing technique where the tester checks the functionality of the software without knowing its internal code or logic.

- **Equivalence Partitioning:** Dividing input data into distinct classes and testing a single value from each class to ensure broad coverage.

Example:

Age field allowed: 18–60

Valid: 25

Invalid: 10, 65

- **Boundary Value Analysis (BVA):** Focusing tests on the boundary values of valid and invalid input ranges, where errors are most likely to occur.

Example:

Range: 1–100

Test: 0, 1, 2, 99, 100, 101

- **Decision Table Testing:** Using a table to test all possible combinations of inputs and the resulting outputs for complex business rules.

Example: Login allowed only if:

Username valid ✓

Password valid ✓

- **Use Case Testing:** Designing test cases based on user scenarios and end-user interactions to cover business flows.

Example:

“User registers → logs in → updates profile → logs out”

2. White-Box Techniques

White Box Testing is a software testing technique in which the tester knows the internal code, logic, and structure of the program and designs test cases to verify them.

- **Statement Coverage:** Ensuring every executable statement in the source code is run at least once.

- **Decision (Branch) Coverage:** Ensuring all branches from each decision point (e.g., if/else statements) are executed to test both outcomes.

3. Experience-Based Techniques

Experience-Based Testing is a testing approach where test cases are designed based on the tester's knowledge, skills, and past experience, rather than formal requirements or test design techniques.

- **Error Guessing:** Testers predict potential defects based on their experience and intuition to create targeted test cases.
- **Exploratory Testing:** Designing and executing tests simultaneously, often without formal documentation, to discover unexpected behavior.

Benefits of test case design:

- **Improved Quality & Reliability:** Detects defects early, validates features against requirements, and ensures a seamless user experience, leading to a more robust product.
- **Cost & Time Efficiency:** Reduces debugging efforts and future maintenance costs by catching bugs sooner; well-designed tests are reusable and support automation, saving time and effort.
- **Comprehensive Coverage:** Techniques like Boundary Value Analysis and Equivalence partitioning ensure all critical paths and edge cases are tested, preventing gaps.
- **Better Documentation & Collaboration:** Creates clear, standardized documentation that aligns teams, clarifies requirements, and serves as a knowledge base for future development and maintenance.
- **Customer Satisfaction:** Delivers software that meets user expectations and functions as intended, increasing trust and driving profits.
- **Supports Automation:** Well-structured, scripted test cases are ideal for automation, enabling continuous integration and faster feedback loops.

❖ SOFTWARE TESTING STRATEGIES

I. White box testing

Using white-box testing methods, the software engineer can derive test cases that:

- Guarantee that all independent paths within a module have been exercised at least once,
- Exercise all logical decisions on their true and false sides,
- Execute all loops at their boundaries and within their operational bounds, and
- Exercise internal data structures to ensure their validity

The main Reasons for conducting white-box tests can be listed as:

- Logic errors and incorrect assumptions are inversely proportional to the probability that a program path will be executed. Errors tend to creep into our work when we design and implement function, conditions, or controls that are out of the mainstream. Everyday processing tends to be well understood (and well scrutinized), while "special case" processing tends to fall into the cracks.
- We often believe that a logical path is not likely to be executed when, in fact, it may be executed on a regular basis. The logical flow of a program is sometimes counterintuitive, meaning that our unconscious assumptions about flow of control and data may lead us to make design errors that are uncovered only once path testing commences.
- Typographical errors are random. When a program is translated into programming language source code, it is likely that some typing errors will occur.

White box testing technique:

a. Basis path testing

Basis Path Testing is a white-box testing technique that ensures all independent execution paths in a program are tested at least once. It is white-box testing technique first proposed by Tom McCabe. The basis path method enables test case designer to derive a logical complexity measure of a procedural design and use this measure as a guide for defining a basis set of execution paths. Test cases derived to exercise the basis set are guaranteed to execute every statement in the program at least one time during testing. The basics path testing can be done by using the terms: *Flow Graph Notation, Cyclomatic Complexity, Deriving Test Cases, Graph Matrices*

Four steps of basis path:

1. Develop the control flow graph(CFG)
2. Determine the cyclomatic complexity $V(G)$.

Cyclomatic Complexity is computed in one of three ways:

- The number of regions of the flow graph corresponds to the cyclomatic complexity. i.e. $V(G) = R$
- Cyclomatic complexity, $V(G)$, for a flow graph, G , is defined as $V(G) = E - N + 2$, where E is the number of flow graph edges; N is the number of flow graph nodes.
- Cyclomatic complexity, $V(G)$, for a flow graph, G , is also defined as $V(G) = P + 1$, where P is the number of predicate nodes contained in the flow graph G .

3. Identify the independent path or basis path.

4. Design test case for each basis path.

Cyclomatic Complexity is a software metric that measures the logical complexity of a program by determining the number of independent execution paths in its source code. **Cyclomatic complexity** has a foundation in graph theory and provides us with extremely useful software metric.

Benefits of cyclomatic complexity :

1. Reduce the coupling of the code. The higher the cyclomatic complexity number, the more coupled the code is. Highly coupled code cannot be modified easily and independently of other code.
2. Ease of understanding the code increases as the complexity decreases. With a higher complexity number, the programmer has to deal with more control path in the code which leads to more unexpected results and defects.
3. Ease of testing. If a method has a cyclomatic complexity of 10, it means there are 10 independent paths through the method. This implies that at least 10 test cases are needed to test all the different path through the code. The lesser the no, the easier it is to test.

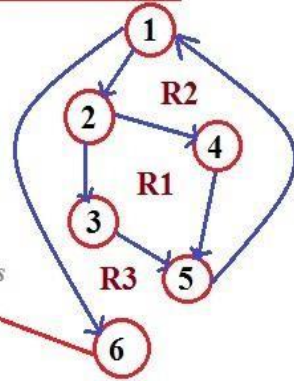
Example

1. Given Code

```
① While (x < y)
{
  ② If (x == 3) then
    Print ("..."); ③
  Else
    ④ Print ("...");
  ⑤ }
  X++; ⑥
```

The terminating point is necessary so it is

2. Flow graph



3. Cyclomatic complexity V(G)

a) Here,

Number of region (R) = 3

Thus, $V(G) = R = 3$

b) Again,

Number of Edges (E) = 7

Number of Nodes (N) = 6

Thus, $V(G) = E - N + 2 = 7 - 6 + 2 = 3$

C) Also,

The number of Predicate nodes (P) = 2 (i.e. node 1 & 2)

Thus, $V(G) = P + 1 = 2 + 1 = 3$

4. Basic paths

- Path 1: 1 – 6
- Path 2: 1 – 2 – 3 – 5 – 1 – 6
- Path 3: 1 – 2 – 4 – 5 – 1 – 6

5) Test Cases

- For Path: 1 – 6
A test case is: $X = 4$ & $Y = 3$
- For Path: 1 – 2 – 3 – 5 – 1 – 6
A test case is: $X = 3$
- For Path: 1 – 2 – 3 – 5 – 1 – 6
A test case is: $X = 2$ or any other except 3

Note:

- Each & every nodes must be terminating on one node so we need the node – 6 on the given example.
- The **Cyclomatic complexity** must be equal in all these three cases.
- If there are other paths beside the **Basic path** then they are redundant path.
- The **test case design** has the highest probability of finding the most errors with a *minimum amount of time and effort*.

b. Control structure testing

The basis path testing technique is one of a number of techniques for control structure testing but it is not sufficient on control structure testing. Control Structure Testing is a white-box testing technique that focuses on testing the logical flow of control in a program. It verifies sequence, selection & iterations.

Some popular control structure testing techniques are described below:

- **Condition Testing**

Condition testing is a test case design method, which ensures that the logical condition and decision statements are free from errors. A relational expression takes the form:

$E1 <\text{relational-operator}> E2$

Where $E1$ and $E2$ are arithmetic expressions and $<\text{relational-operator}>$ is one of the following: $<$, $\leq$, $=$, $\neq$, $>$, or $\geq$. A *compound condition* is composed of two or more simple conditions, Boolean operators, and parentheses. We assume that Boolean operators allowed in a compound condition include OR ($|$), AND ($\&$), and NOT ($\neg$). A condition without relational expressions is referred to as a Boolean expression.

- **Data Flow Testing**

The data flow testing method selects test paths of a program according to the locations of definitions and uses of variables in the program. To illustrate the data flow testing approach, assume that each statement in a program is assigned a unique statement number and that each function does not modify its parameters or global variables. For a statement with S as its statement number,

$DEF(S) = \{X \mid \text{statement } S \text{ contains a definition of } X\}$ $USE(S) = \{X \mid \text{statement } S \text{ contains a use of } X\}$

If statement S is an *if* or *loop* statement, its DEF set is empty and its USE set is based on the condition of statement S . The definition of variable X at statement S is said to be *live* at statement S' if there exists a path from statement S to statement S' that contains no other definition of X .

• Loop Testing

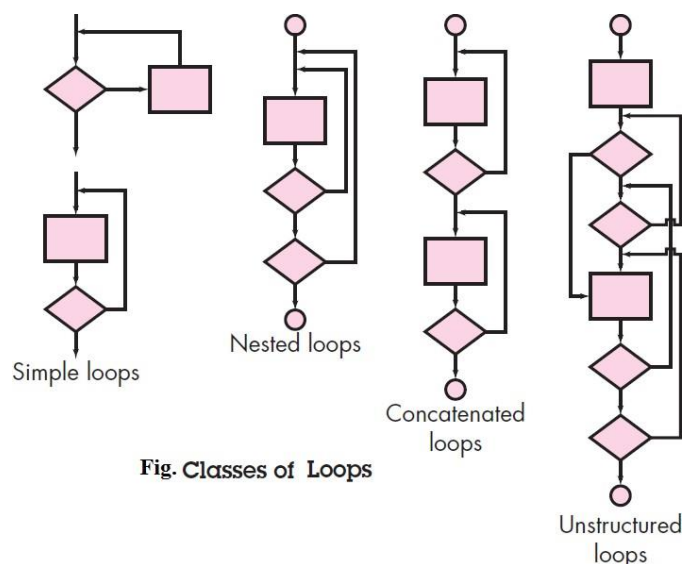
Loops are the cornerstone for the vast majority of all algorithms implemented in software. And yet, we often pay them little heed while conducting software tests. *Loop testing* is a white-box testing technique that focuses exclusively on the validity of loop constructs. Four different classes of loops can be defined: simple loops, concatenated loops, nested loops, and unstructured loops.

Simple loops: They are sequentially tested for the data of desired domain.

Nested loop: They are started to test from innermost loop.

Concatenated loop: Multiple loops are tested simultaneously if there is no dependency exists.

Unstructured loops: Whenever possible, this class of loops should be redesigned to reflect the use of the structured programming constructs.



Advantages of White Box Testing

- Thorough testing of all code and logic.
- Helps in code optimization and removing redundant code.
- Detects defects early in development.
- Fits well with the Software Development Life Cycle (SDLC).
- Detects complex defects not found by other methods.
- Allows creation of comprehensive test cases covering all paths.

Disadvantages of White Box Testing

- Requires programming knowledge and access to source code.
- May overemphasize internal code and ignore functionality.
- Tester bias due to familiarity with the code.
- Test cases must be updated if code changes.
- Cannot detect missing functionalities.
- Time-consuming and requires skilled testers.

II. Black-box testing

Black Box Testing is a software testing technique where the tester verifies the functionality of the application without knowing its internal code or structure. The focus is on input and expected output.

Black box testing techniques:

a. Equivalence partitioning

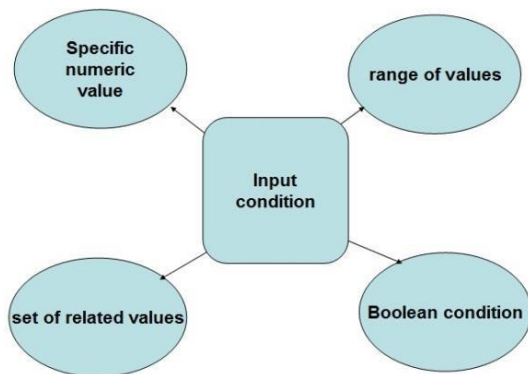
Divides the input domain of a program into classes of data from which test cases can be derived. An equivalence class represents a set of valid or invalid states for input conditions. Equivalence classes may be defined according to the following guidelines:

Input condition specifies a range, one valid and two invalid equivalence classes are defined.

Input condition needs specific value, one valid and two invalid equivalence classes defined.

Input condition specifies member of a set, one valid/one invalid equivalence class defined.

Input condition is Boolean, one valid and one invalid class are defined.



The input conditions associated with each data element for the banking application can be specified as:

- i. **area code:** Input condition, *Boolean*-area code may/may not be present.
Input condition, *range*—values defined between 200 and 999, with specific exceptions.
- ii. **prefix:** Input condition, *range*—specified value >200
- iii. **suffix:** Input condition, *value*—four-digit length
- iv. **password:** Input condition, *Boolean*-password may/may not be present.
Input condition, *value*—six-character string.
- v. **command:** Input condition, *set*-containing commands noted previously.

b. Boundary value analysis

A greater number of errors tend to occur at the boundaries of the input domain rather than in the center. Boundary value analysis leads to a selection of test cases that exercise bounding values. BVA leads to the selection of test cases at the *edges* of the class. The boundary value analysis can be decomposing in following different prospective:

Testing GUIs

Finite state modeling graphs may be used to derive a series of tests that address specific data and program objects that are relevant to the GUI. It helps make sure that all possible actions a user can take are tested, and the application behaves correctly.

Testing of Client/Server Architectures

The major difficulties in testing of C/S architectures and the software can be listed as:

- The distributed nature of client/server environments
- The performance issues associated with transaction processing,

- The potential presence of a number of different hardware platforms,
- The complexities of network communication
- The need to service multiple clients from a centralized or distributed database, the coordination requirements imposed on the server all combine to make

Testing for Real-Time Systems

The time-dependent, asynchronous nature of many Real-time applications adds a new and potentially difficult element to the testing mixes—time. Not only does the test case designer have to consider white- and black-box test cases but also event handling (interrupt processing), the timing of the data, and the parallelism of the tasks (processes) that handle the data. Four step strategy for testing Real-time Systems: Task testing, Behavioral testing, Inter-task testing, System Testing.

c. Decision Table Testing

Decision table testing is a systematic and organized black box testing technique used to deal with complex systems. This technique is beneficial when the system's behavior is different for different combinations of inputs. It's often used when there are multiple inputs that can have different values and can result in different outputs.

A decision table represents inputs and outputs in a structured, tabular form, making it easy to understand. It ensures the coverage of all possible combinations, thus making the test cases more comprehensive and robust.

d. State Transition Testing

State transition testing is a black box testing technique used to test the behavior of a software application for different input conditions given in a sequence. It's particularly useful when software behavior changes from one state to another following particular actions.

This technique uses a state transition diagram to represent the different states of a system and the transitions from one state to another. It can help validate a software application's behavior when it has a sequence of events or needs to maintain a specific order of events.

e. Use Case Testing

Use case testing is a black box testing technique that uses use cases to identify test cases. A use case is a description of a system's behavior as it responds to an end-user's need or request.

This testing technique helps in identifying all possible scenarios for a particular functionality. It ensures that the system can handle and respond to every request correctly and effectively.

Advantages of Black Box Testing

- The tester does not need to have more functional knowledge or programming skills to implement the Black Box Testing.
- It is efficient for implementing the tests in the larger system.
- Tests are executed from the user's or client's point of view.
- Test cases are easily reproducible.
- It is used to find the ambiguity and contradictions in the functional specifications.

Disadvantages of Black Box Testing

- There is a possibility of repeating the same tests while implementing the testing process.
- Without clear functional specifications, test cases are difficult to implement.
- It is difficult to execute the test cases because of complex inputs at different stages of testing.
- Sometimes, the reason for the test failure cannot be detected.
- Some programs in the application are not tested.
- It does not reveal the errors in the control structure.
- Working with a large sample space of inputs can be exhaustive and consumes a lot of time.

White Box Testing	Black Box Testing
1. It concerned on structural testing of module which explains the internal logic of a program and particular execution path. 2. White box testing is performed early in the testing process (<i>unit testing</i>). 3. It is also known as "<i>glass testing</i>", "<i>Clear box testing</i>" or "<i>Structural Testing</i>" as it specify the control structure. 4. White Box Testing require the programming knowledge to test the modules as it tester must correct the error codes. 5. It will be effective for small line or piece of codes. 6. Here, we have to test the logic with specifying the large quantity of test cases. <u>1. Given Code</u> <pre> ① While (x < y) { ② If (x == 3) then Print ("..."); ③ Else ④ Print ("..."); ⑤ X++; ⑥ } </pre> The terminating point is necessary so it is <u>2. Flow graph</u>	1. It determines the functional requirement of the software that involves the input conditions, interface errors, error in data structure etc. 2. Black box testing tends to be applied during the later stage of testing (<i>integration & system testing</i>) because it focuses on the information domain. 3. It concerns to the functional performance so it is also known as "<i>Behavioral testing</i>". 4. Here, the application can be tested without programming knowledge, so only external functional behaviors & GUI features are examined. 5. We can use the Black Box Testing for any small or large module. 6. It can be done by selecting simple test cases such as particular input is given and the prescribed output is examined.

5.8 SOFTWARE MAINTENANCE AND ITS TYPES:

Software Maintenance refers to the process of modifying and updating a software system after it has been delivered to the customer. This involves fixing bugs, adding new features, and adapting to new hardware or software environments. Effective maintenance is crucial for extending the software's lifespan and aligning it with evolving user needs.

The goal of software maintenance is to keep the software system working correctly, efficiently, and securely, and to ensure that it continues to meet the needs of the users.

This can include fixing bugs, adding new features, improving performance, or updating the software to work with new hardware or software systems. It is also important to consider the cost and effort required for software maintenance when planning and developing a software system.

Several Key Aspects of Software Maintenance:

1. **Bug Fixing:** The process of finding and fixing errors and problems in the software.
2. **Enhancements:** The process of adding new features or improving existing features to meet the evolving needs of the users.
3. **Performance Optimization:** The process of improving the speed, efficiency, and reliability of the software.
4. **Porting and Migration:** The process of adapting the software to run on new hardware or software platforms.
5. **Re-Engineering:** The process of improving the design and architecture of the software to make it more maintainable and scalable.
6. **Documentation:** The process of creating, updating, and maintaining the documentation for the software, including user manuals, technical specifications, and design documents.

Types of Software Maintenance:

1. Corrective maintenance:

Corrective maintenance is the type of software maintenance that involves identifying and fixing errors, defects, or faults in a software system after it has been delivered and is in use, or to enhance the performance of the system. The purpose of corrective maintenance is to restore systems that have broken down.

2. Adaptive maintenance:

This includes modifications and updation when the customers need the product to run on new platforms, on new operating systems, or when they need the product to interface with new hardware and software.

3. **Perfective maintenance:**

A software product needs maintenance to support the new features that the users want or to change different types of functionalities of the system according to the customer's demands.

4. **Preventive maintenance:**

This type of maintenance includes modifications and updations to prevent future problems with the software. It goals to attend to problems, which are not significant at this moment but may cause serious issues in the future.

Benefits of Software Maintenance

- **Reliability & Stability:** Fixes bugs and errors, reducing crashes and downtime.
- **Security:** Patches vulnerabilities to protect data and systems.
- **Performance:** Improves speed and efficiency, enhancing user experience.
- **Adaptability:** Ensures software works with new systems and changing requirements.
- **User Satisfaction:** Enhances functionality and usability for users.
- **Extends Lifespan:** Keeps software relevant and prevents obsolescence.
- **Cost Savings:** Prevents major breakdowns and reduces expensive fixes.
- **Compliance:** Aligns software with legal and regulatory standards.

ASSIGNMENTS

1. Solve the all Cyclomatic complexity numerical from past year questions.
2. Define equivalent partitioning testing. Perform the equivalent partitioning testing of input field named age which takes range of value between 20 to 50.
3. Define cyclomatic complexity. Write down the four steps of basis path testing.
4. What is black box testing? Black box vs white box testing.
5. Define system testing. explain about unit testing and importance of drivers and stubs during unit testing.
6. Define verification and validation. Mention the reason for conducting black box testing.
7. Explain the concepts of unit testing and regression testing in software development. Discuss their significance in ensuring software quality.
8. Define boundary value analysis. Difference between alpha & beta testing.
9. Define software maintenance. Explain its types.

QN-5 SOLUTION:

❖ Importance of drivers and stubs during unit testing:

★ Stubs and Drivers:

Stubs and drivers are temporary modules that act as substitutes for real modules that are missing, under development, or not yet implemented. They are used to satisfy the dependency of the module under test. Stubs and drivers simulate the behavior, features, and functionalities of the actual modules they replace. By doing so, they eliminate unnecessary delays in testing, support parallel development, and make the testing process faster and more efficient.

A stub is a temporary software module used during top-down integration testing to replace lower-level modules that are not yet implemented. Stubs simulate the behavior of the called modules and provide minimal functionality required for testing higher-level modules.

Stubs are used when lower-level modules are missing, under development, or unavailable. They do not perform full processing but return predefined values or display messages so that the calling module can continue execution. The main purpose of stubs is to allow early testing of high-level modules without waiting for the completion of lower-level modules.

Scenario:

- Module to test: ProcessTransaction()
- Dependent module: CalculateInterest() (not ready)

A driver is a temporary software module used during **bottom-up integration testing** to replace higher-level modules that are not yet developed. A driver simulates the calling module and controls the execution of the lower-level module under test.

Drivers provide test inputs to the module being tested and collect outputs for verification. They are generally more complex than stubs because they perform the role of a main control program.

Drivers are used when higher-level modules are unavailable, allowing lower-level modules to be tested independently until full integration is possible.

Scenario:

- Module to test: CalculateInterest(amount)
- Higher-level module: ProcessTransaction() (not ready)

Lower-level modules usually **don't run on their own** — they are designed to be **called by higher-level modules**.

Importance of stub and driver during unit testing:

1. Enable Independent Testing of a Module

- Unit testing focuses on one module at a time.
- If the module depends on other modules that are not ready, **stubs (for lower-level modules)** or **drivers (for higher-level modules)** allow the unit to run and be tested independently.

2. Simulate Missing Modules

- They **mimic the behavior** of the unavailable modules by returning dummy values (stubs) or providing input (drivers).
 - This ensures that testing of the module under test can proceed without waiting for all modules to be completed.
3. **Support Early Detection of Errors**
 - By testing a module even when dependencies are missing, errors in the module itself can be detected early, saving time and cost.
 4. **Facilitate Parallel Development**
 - Development and testing of different modules can happen **simultaneously**, because the unavailable modules are simulated using stubs or drivers.
 5. **Verify Module Logic and Flow**
 - Even with dummy data, the module can be tested for:
 - Correct calculations
 - Conditional logic
 - Data handling
 - Ensures the module behaves as expected in isolation.